MABEL Arm Control: Kinematics, Inverse Dynamics, Gravity Compensation, and Back-Drivable Compliance

MABEL Simulation Project*

June 8, 2026

Abstract

This document specifies the arm-level control of MABEL and the compliant ("drag-the-robot") whole-body mode, at the level of a methods section. It covers (i) the manipulator kinematics — frames, a Denavit–Hartenberg (DH) parameterization, forward kinematics, and the geometric Jacobian; (ii) the manipulator dynamics — the equations of motion and their recursive Newton–Euler evaluation; and (iii) *gravity compensation* and *back-drivable compliant control*, by which cancelling the modelled bias torque turns each joint into a pure inertia that a human can move by hand, with the torso and lift made deliberately "lazy" and (next step) the mobile base following the inferred drag. The controller is *model-based* with a swappable plant, so the same code runs in simulation and on hardware, and is *adaptable* to the real robot's uncertain, changing link masses and centres of mass. The bimanual task-space inverse kinematics is the subject of `whole_body_control.pdf` and the mobile base of `motion_model.pdf`; both are referenced, not repeated. Implementation: `mabel/dynamics.py`, `gravity_compensation.py`.

Keywords: Gravity compensation, inverse dynamics, back-drivability, manipulator kinematics, physical human–robot interaction.

1 Manipulator kinematics

Each arm is a 7-revolute (7R) chain terminated by a roll wrist and the dexterous hand; MABEL is bimanual (left/right). We describe the left arm; the right is its mirror.

1.1 Joints and frames

Table 1 lists the joints as defined in `models/mabel_full.xml`: each successive link frame rotates about its local z axis (the wrist about $-x$), with the constant link offsets carried by the body frames. The seven proximal joints give a redundant positioning/orienting arm; the wrist adds a forearm roll.

*Companion notes: base motion (`motion_model.pdf`), whole-body IK (`whole_body_control.pdf`), compliance study (`compliance_study.pdf`).

Joint i	axis (local)	range [rad]	motor
Arm 1	z	$[-3.49,\ 1.40]$	DM-J8009
Arm 2	z	$[-0.17,\ 3.32]$	DM-J8009
Arm 3	z	$[-1.57,\ 1.57]$	DM-J4340
Arm 4	z	$[-0.77,\ 1.67]$	DM-J4340
Arm 5	z	$[-1.57,\ 1.57]$	DM-J4310
Arm 6	z	$[-0.79,\ 0.79]$	DM-J4310
Arm 7	z	$[-1.57,\ 1.57]$	DM-J4310
Wrist	$-x$	$[-0.90,\ 0.67]$	DM-J4310

Table 1: Left-arm joint axes, limits, and actuators (from `mabel_full.xml`). The right arm mirrors these. Joint limits are enforced by clamping in the controllers; see `whole_body_control.pdf` §4.4.

1.2 Denavit–Hartenberg parameterization and forward kinematics

Adopting the standard DH convention, consecutive link frames are related by four parameters $(\theta_i, d_i, a_i, \alpha_i)$ — joint angle, link offset, link length, and twist — through the homogeneous transform

$$ {}^{i-1}T_i(\theta_i) = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{1} $$

($c = \cos, s = \sin$), and the forward kinematics of the palm in the arm-base frame is the product

$$ {}^{0}T_{\text{palm}}(q) = \prod_{i=1}^{8} {}^{i-1}T_i(\theta_i). \tag{2} $$

Note on the source of the parameters. MABEL is authored in MuJoCo, where the kinematics is specified directly by each body's frame (`pos`, `quat`) and joint `axis` rather than as a DH table; the equivalent DH parameters are obtained by aligning each z_i with joint axis i and applying the usual common-normal construction to (1). We therefore *use* the MJCF frames as the ground truth for FK (MuJoCo's `mj_forward` realizes (2)) and treat the DH table as the analytic equivalent; only the joint variables θ_i change at run time, the link constants do not.

1.3 Geometric Jacobian

The palm spatial velocity (twist) relates to joint rates by the geometric Jacobian $J(q) \in \mathbb{R}^{6\times 8}$,

$$ \begin{bmatrix} v_{\text{palm}} \\ \omega_{\text{palm}} \end{bmatrix} = J(q)\,\dot{q}, \qquad J = \begin{bmatrix} J_1 & \cdots & J_8 \end{bmatrix}, \quad J_i = \begin{bmatrix} z_i \times (p_{\text{palm}} - p_i) \\ z_i \end{bmatrix}, \tag{3} $$

with z_i the world axis and p_i the origin of joint i (all revolute). In code this is queried as the MuJoCo body Jacobian `mj_jacBody`. The arm is kinematically redundant for a 6-DOF palm task ($8 > 6$), and that redundancy is exploited by the null-space posture term of the task-space controller.

1.4 Inverse kinematics (reference)

Palm pose targets are tracked by the bimanual SE(3) differential IK — weighted damped least squares with a null-space posture task — derived in full in `whole_body_control.pdf`. At the arm

level it reads $\dot{q} = J^{\#}\,\xi + N\,\dot{q}_0$ with the damped weighted pseudo-inverse $J^{\#}$ of (3) and null-space projector $N = I - J^{\#}J$; we do not repeat that derivation here.

2 Manipulator dynamics

2.1 Equations of motion

The arm (indeed the whole body) obeys the rigid-body equations of motion

$$M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + g(q) \;=\; \tau + \tau_{\text{ext}}, \tag{4}$$

where $M(q) \succ 0$ is the joint-space inertia matrix, $C(q,\dot{q})\dot{q}$ the Coriolis/centrifugal vector, $g(q)$ the gravity vector, τ the actuator torques, and $\tau_{\text{ext}} = J^{\top} F_{\text{ext}}$ the joint torques from an external wrench F_{ext} (e.g. a human hand). We write the *bias* as $b(q,\dot{q}) = C(q,\dot{q})\dot{q} + g(q)$.

2.2 Recursive Newton–Euler evaluation

Rather than forming M, C, g explicitly we evaluate the inverse dynamics $\tau = \mathrm{ID}(q,\dot{q},\ddot{q}) = M(q)\ddot{q} + b(q,\dot{q})$ by the recursive Newton–Euler algorithm (RNE), $O(n)$ in the number of links. The implementation (`mabel/dynamics.py`) exposes three queries:

$$\text{gravity:}\quad g(q) = \mathrm{ID}(q,0,0), \tag{5}$$

$$\text{bias:}\quad b(q,\dot{q}) = \mathrm{ID}(q,\dot{q},0), \tag{6}$$

$$\text{full ID:}\quad \tau = \mathrm{ID}(q,\dot{q},\ddot{q}), \tag{7}$$

exposed as `UpperBodyDynamics.gravity` (RNE at $\dot{q},\ddot{q} = 0$), `.bias` (MuJoCo `qfrc_bias`), and `.inverse_dynamics`.

3 Gravity compensation and back-drivable compliance

3.1 Static gravity compensation

Commanding $\tau = \hat{g}(q)$ cancels the modelled gravity load so the arm neither sags nor must be held by stiff servos:

$$\boxed{\tau_{\text{gc}} = \hat{g}(q).} \tag{8}$$

With perfect $\hat{g} = g$ and $\dot{q} \approx 0$, (4) reduces to $M\ddot{q} + C\dot{q} = \tau_{\text{ext}}$: the arm holds any configuration and yields to the slightest external force.

3.2 Full-bias cancellation ⇒ pure inertia

For motion (not just statics) we cancel the *full* bias and add a small viscous term $D \succeq 0$:

$$\boxed{\tau = b(q,\dot{q}) - D\,\dot{q}.} \tag{9}$$

Substituting into (4) and using $b = C\dot{q} + g$ gives the closed-loop behaviour

$$M(q)\,\ddot{q} + D\,\dot{q} = \tau_{\text{ext}} = J^{\top} F_{\text{ext}}, \tag{10}$$

i.e. the joint behaves as its *own inertia with damping D, gravity- and Coriolis-free*. This is exactly back-drivability: a person pushing the link with wrench F_{ext} moves it as a damped mass, and it coasts to rest when released. D trades drift (numerical / model error) against drag effort: small D on the arms (light, free), large D on the "lazy" joints (§4.1).

Critical note (model error). The human feels the *residual* $\tilde{b} = b - \hat{b}$ from inertial-parameter error. Gravity dominates it, so the fidelity of $\hat{g}$ — i.e. of the link masses and CoMs — sets how "weightless" the arm feels; see §4.4. Joint friction (not modelled by RNE) also appears as residual resistance and may warrant a feed-forward friction term on hardware.

3.3 Compliance controller

`ComplianceController` (`mabel/dynamics.py`) implements (9) per tick: it neutralizes the stiff position servo on each managed joint (so only the computed torque acts) and applies $b_i - D_i\dot{q}_i$. Validation in simulation: with the servos disabled and *no* compensation the arm sags >1 rad in 3 s; with (9) it holds to <0.1 rad and a 0.5 N m nudge visibly moves it.

4 Compliant whole-body control (drag-to-move)

4.1 Lazy torso and lift

The torso and lift use the same law (9) but with a large damping $D_{\text{lazy}} \gg D_{\text{arm}}$: gravity-held (they do not drop) yet high apparent viscosity, so they move *slowly* and only under sustained force. The lift is tendon-driven (cascaded); its servo is likewise neutralized and the compensation applied at the lift joint DOF.

4.2 Base follows the drag (next step)

The goal is that pulling an arm drags the whole robot. With the arms compliant, the operator's intent appears as the external wrench F_{ext} (or, equivalently, the drift of the palm targets / measured palm velocity). The mobile base is then commanded a body twist aligned with that intent,

$$(v_x, v_y, \omega)_{\text{base}} \;=\; K_{\text{drag}}\, \Pi\big(\hat{v}_{\text{hands}}\big), \tag{11}$$

where $\hat{v}_{\text{hands}}$ is the estimated hand velocity/force direction and Π projects it to the planar base. The base is made *very agile* for this mode (high K_{drag}, low smoothing) so it offloads the arms before they hit their workspace limits — the active analogue of the reactive "lazy base" of `whole_body_control.pdf` §6, here driven by human intent rather than reach. Execution still passes through the swerve IK and the tip-over limits of `motion_model.pdf`. (Implemented in a subsequent step on top of the compliance layer of this document.)

4.3 Modularity: swapping simulation for hardware

The dynamics model and the plant are separated:

Role	Class	Sim / Real
torque model	`UpperBodyDynamics`	MuJoCo model (both; fed measured $q, \dot{q}$ on HW)
state & effort I/O	`Plant` (ABC)	`SimPlant` (`qfrc_applied`) / `RealPlant` (CAN torque)
control law	`ComplianceController`	identical

On hardware the same `UpperBodyDynamics` computes $\hat{g}, \hat{b}$ from measured joint states (model-based control), and a `RealPlant` sends the torques to the DAMIAO drivers in MIT/torque mode; the controller code is unchanged.

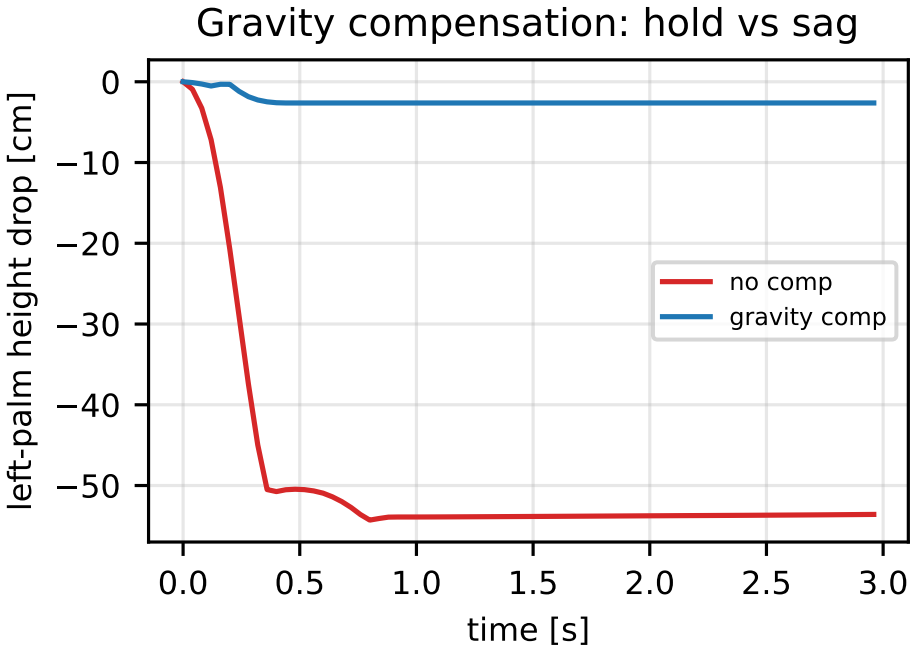


Figure 1: Gravity compensation: left-palm height vs. time with servos disabled. Uncompensated the arm sags; compensated it holds, confirming (8).

4.4 Adapting to changing mass and CoM

Because $g(q)$ and $b(q, \dot{q})$ are functions of the link masses m_i and CoMs c_i, the controller adapts simply by updating those in the model: `set_link_mass`, `set_link_com`, `scale_total_mass` (followed by `mj_setConst` to refresh derived quantities). This covers a tool/payload change or a re-estimated chassis. Since the residual felt by the operator is $\tilde{g} = g(m, c) - \hat{g}(\hat{m}, \hat{c})$, these parameters can also be *fit online*: command (8), hold still, and regress the measured holding torque (linear in the inertial parameters) to refine $\hat{m}, \hat{c}$ — the standard payload-identification step, left as a hook here.

5 Experiments

We verify the gravity-compensation layer in MuJoCo (`tools/doc_experiments.py`, `tools/compliance_study.py`).

5.1 Holding under gravity

Fig. 1 tracks the left-palm height with the arm servos disabled. Without compensation the arm collapses (>1 m within 1 s); with the bias torque (9) it holds within a centimetre, i.e. the modelled gravity is cancelled and the arm is free yet self-supporting.

5.2 Back-drivability and parameter robustness

Fig. 2(a) shows the residual generalized force the operator feels remains <0.06 N m even at $\pm 40\%$ mass-model error—compensation degrades gracefully, supporting the run-time mass adaptation of

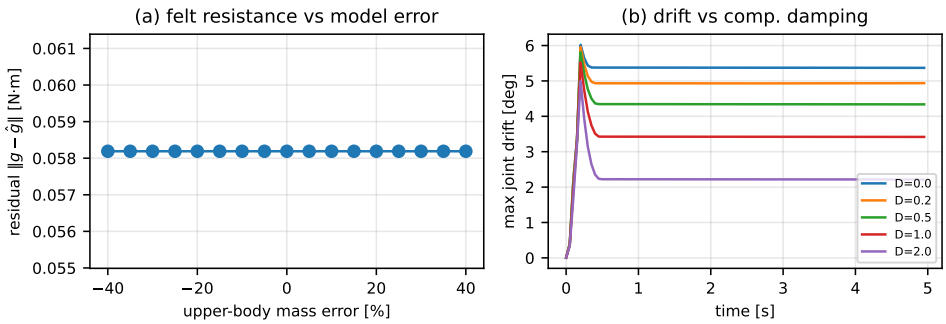


Figure 2: (a) Felt residual torque vs. mass-model error. (b) Free-float drift vs. compensation damping. The arm stays back-drivable across a wide payload error.

§4.4. Fig. 2(b) shows the free-float joint drift vs. the compensation damping D: D trades a light feel against drift, with $D \approx 0.6\,\mathrm{N\,m\,s}$ a good compromise.

6 Assumptions & limitations

- **Model-based.** Compliance quality is set by the inertial-parameter fidelity; gravity error is felt directly (§4.4).
- **No joint friction / stiction model.** RNE omits friction; on hardware add a feed-forward friction term, else it reads as residual drag.
- **Ideal torque actuation.** (9) assumes the drivers realize commanded torque (DAMIAO MIT mode); current/torque-tracking bandwidth and limits are not modelled here.
- **Damping vs. drift.** $D = 0$ is ideal compliance but drifts under model/integration error; a small D is used in practice.
- **Base follower is future work** (§5.2); this document delivers the ID + gravity-compensation + compliance layer it builds on.